/Github/

Flora Belas

/O que é?/

- O **GitHub** é uma plataforma online de hospedagem de código-fonte e colaboração, baseada no sistema de controle de versão **Git**.
- **Git** é um software de linha de comando mantido pela Linux Foundation, focada no versionamento técnico do código.
- **GitHub** é um serviço online de propriedade da Microsoft que oferece interface gráfica, recursos de rede social, autenticação e ferramentas de gerenciamento para hospedar os repositórios do Git. GitHub é o site/serviço em nuvem onde você hospeda e compartilha os repositórios criados com Git



/Git vs Github/



O **Git** é um sistema de controle de versão **local** (roda no seu computador, não online) que rastreia alterações em arquivos e permite trabalhar offline.

O **GitHub** é uma plataforma de hospedagem baseada na nuvem que facilita a colaboração online e o gerenciamento de projetos.

/Git/

- **Git** acompanha toda mudança que você faz nos seus arquivos o tempo todo. Ele acompanha **o que mudou, quando mudou, quem mudou, onde mudou**. Linha a linha.
- Isso inclui **qualquer tipo de arquivo**. Uma imagem, um arquivo de texto, um script python, até mesmo um vídeo.

- Ele permite você manter diversas versões do mesmo arquivo, e a qualquer momento você pode voltar a uma versão anterior.

/Local vs Remote/

- **Local** se refere ao seu computador, onde você programa e faz suas alterações.
- Remote se refere a nuvem (na internet), para onde você manda os seus arquivos locais. Na maior parte dos casos remote significa o Github.

/Github/

- Funciona como o ponto central para projetos de um time. Cada computador de um time tem sua versão dos arquivos, usando o Git, e todos enviam suas modificações para o **Github**, fazendo um **merge**.
- Isso inclui **qualquer tipo de arquivo**. Uma imagem, um arquivo de texto, um script python, até mesmo um vídeo.

/Pra que serve?/

- **Hospeda repositórios Git na nuvem**, permitindo que equipes trabalhem no mesmo projeto e mantenham histórico de alterações.
- **Facilita a colaboração** via **pull requests** (revisão e mesclagem de código) e **issues** (rastreio de bugs e tarefas).
- **Oferece automação** (GitHub Actions), documentação, páginas públicas e integrações com IA (GitHub Copilot).
- Funciona como uma rede social/portfólio para desenvolvedores, onde é possível contribuir em projetos open-source e exibir o próprio trabalho.

/Repositório/

- É a "pasta" do projeto hospedada no GitHub. Contém todos os arquivos, o histórico de commits e as branches.

Novo repositório (local)	git init
Novo repositório (no GitHub)	gh repo create meu-repo --public --source=. --push
Clone (baixar do GitHub pro PC)	git clone <url>

/Commit vs Push/

- **Commit** → ponto de salvamento **no seu computador**
- **Push** → manda os commits **pro GitHub**
- São duas coisas, não uma
- Você pode fazer vários commits e um push só no fim
- □ □ **Só comitou? Não aparece no GitHub.**

Você escreve código
↓
COMMIT (Ctrl+K)
↓
está salvo no seu PC
↓
PUSH (Ctrl+Shift+K)
↓
está no GitHub

/Staging/

- O Git tem três lugares: **pasta → fila → histórico**
- **Working dir** são os arquivos da sua branch, onde você edita. É onde você **mexe** e as mudanças ainda **não foram pra lugar nenhum**.
- A **staging area** é a fila do próximo commit. É quando você marca quais arquivos quer commitar.
- git add coloca na fila · git commit registra o que está na fila
- Serve pra escolher o que entra em cada commit

/Staging/

- O Git tem três lugares: **pasta → fila → histórico**
- A staging area é a fila do próximo commit
- git add coloca na fila · git commit registra o que está na fila
- Serve pra escolher o que entra em cada commit

PASTA (working dir)	FILA (staging)	HISTÓRICO (repositório)
Aluna.java	→ git add →	git commit → □
Main.java	→ git add →	git commit → □
teste.txt	(fora da fila, não vai)	

/Merge/

- O Merge é juntar o trabalho de uma branch em outra
- Você fica na branch que vai receber e manda a outra entrar
- É assim que a main recebe código, nunca por commit direto
- Depois do merge, a branch já cumpriu o papel e pode ser apagada
- Apagar a branch após o push (merge é um tipo de push) não apaga o trabalho, os commits já estão na main

/Rebase vs Merge?/

- **O Cenário**: Você baixou o projeto na segunda-feira e começou a codar. Sua colega fez alterações na terça e já subiu para a main. Seu código local ficou com a base desatualizada.

  O Merge: Junta as duas versões criando um nó no histórico (commit de junção), mostrando que vocês trabalharam em paralelo.

  O Rebase: Modifica o histórico e "cola" o seu código no topo da fila, como se você tivesse começado a programar exatamente depois dela.

/.gitignore/

- O **.gitignore** é um arquivo de texto na raiz do seu projeto que diz pro Git: "esses arquivos/pastas, não rastreie, não commita, não manda pro GitHub.“
- É o arquivo que diz o que ser ignorado (não subir pro projeto, não aparece nem no commit).

- Fica na raiz do projeto

/README/

- É um arquivo README.md na raiz do repositório. O GitHub mostra o conteúdo dele automaticamente na página inicial, logo abaixo da lista de arquivos.
- O .md é Markdown — um jeito simples de formatar texto: # vira título, - vira lista, **texto** vira negrito.
- **É o cartão de visitas**. Quem abre seu repositório lê o README antes de olhar qualquer código.

/NUNCA SUBA ISSO/

- Senha, token ou chave de API nunca vão pro GitHub
- Existem robôs que varrem o GitHub procurando chaves
- **Apagar depois não resolve**, fica no histórico pra sempre
- Nos módulos de IA vocês vão usar chaves de acesso. Guardem essa regra desde já.
- A chave fica num arquivo separado, que o .gitignore impede de subir.

```
NUNCA: String apiKey = "sk-proj-abc123xyz...";
```

/Branch/

- Uma linha de desenvolvimento paralela. O repositório sempre começa com uma branch padrão (main). Você cria branches para trabalhar em novas funcionalidades sem afetar o código principal.

Branch (criar)	git branch nome-da-branch
Branch (trocar)	git switch nome-da-branch

/Branch/

- **Nome descritivo** com prefixo: feature/login, fix/bug-pagina-inicial, hotfix/pagamento
- **Uma branch por tarefa** — não misture coisas diferentes
- **Branches curtas** — evite branches que ficam abertas por semanas (aumenta conflito)
- **Nunca commitar direto na main** — sempre via branch + PR

/Branch/

Convenção de nomenclatura:

- feature/nome-da-coisa
- bugfix/nome-do-erro
- hotfix/coisa-urgente

Se vier com id de um card:

- feature/PROJ-123-nome-da-coisa
- bugfix/123-nome-do-erro
- hotfix/BUG-123-coisa-urgente

Ferramenta	Formato típico	Exemplo
Jira	PROJ-123	feature/PROJ-123-aula
Trello	123 (número do card)	feature/123-aula
GitHub Issues	#123 (mas no branch usa só 123)	feature/123-aula

/Pull Request (PR)/

- O coração da colaboração no GitHub. É uma proposta para mesclar as alterações de uma branch em outra (geralmente na main). Permite que outros revisem o código, comentem e aprovem antes do merge.

Push (enviar pro GitHub)	git push
Pull (baixar do GitHub)	git pull

/Pull Request (PR)/

- O commit sempre vai na branch, nunca na main.
- **A main só recebe código por merge.**
- Em time, esse merge passa por Pull Request. Alguém revisa antes.
- Sozinha, você mesma faz o merge. Você é o time.

/Fork/

- Uma cópia completa de um repositório no seu próprio perfil. Usado para contribuir em projetos de terceiros sem ter acesso direto.

Fork (copiar repo de outro pro seu perfil)	gh repo fork dono/repo --clone

/Issues/

Uma issue é um chamado ou título de tarefa dentro do repositório. É como abrir um chamado em um suporte técnico.
Exemplo:

- Você está usando um app e encontra um bug — a tela de login trava.
- Você vai no GitHub do projeto e abre uma issue com título: "Login trava ao digitar a senha".

/Issues/

Issue (criar)	gh issue create
Issue (listar)	gh issue list
Issue (fechar)	gh issue close 42

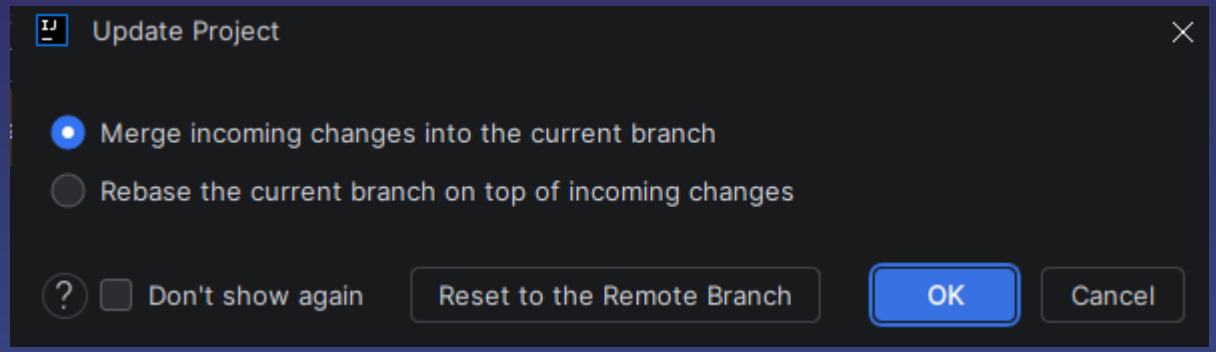
/Fetch/

- Ele vai até o GitHub, baixa os commits/branches **que você ainda não tem**, e atualiza o ponteiro origin/master localmente. **Não mexe em nada do seu trabalho**.
- Mexe apenas no **origin/master**

/Fetch/

- Alguém faz push no GitHub. Com isso o master real do GitHub avança
- Você roda git fetch . Seu **origin/master** local é atualizado pra refletir o novo estado
- Aí sim você pode criar branch a partir de origin/master sabendo que é o mais recente.

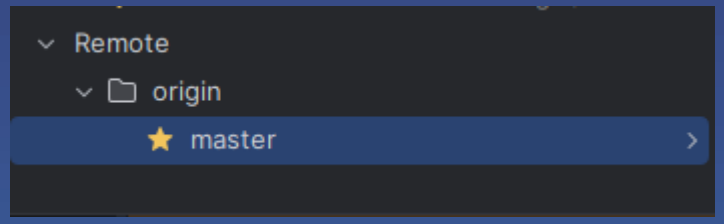
/Update (git pull)/



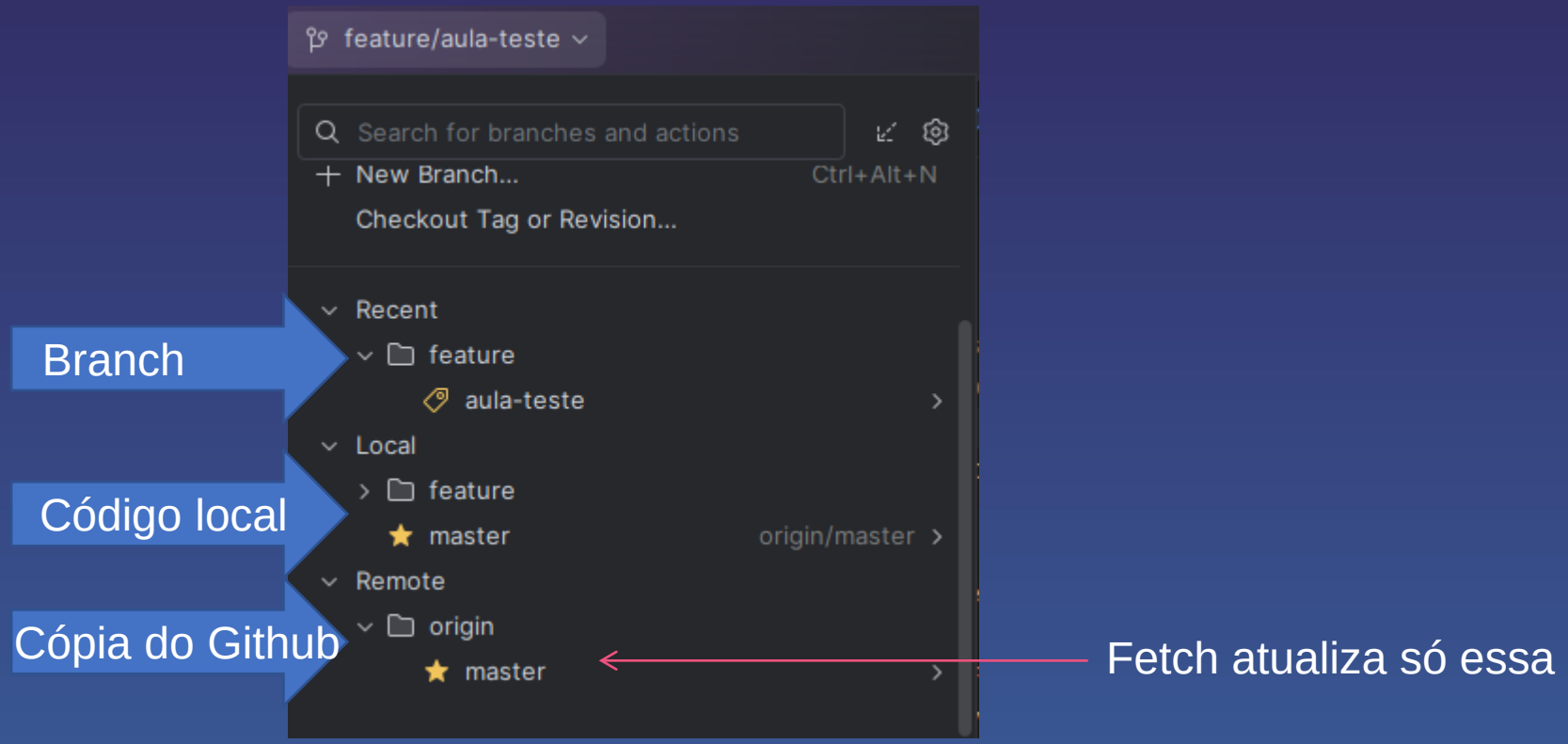
git pull = fetch + merge

/Remote Tracking Branch/

É um "fantasma" do GitHub guardado no seu computador. O seu Git local mantém uma foto em cache de como a branch master estava lá no servidor (origin) na última vez que eles se comunicaram. (origin/master)



/Remote Tracking Branch/



/Checkout/

Use para mudar de branch. Só isso pra gente.

/Principais comandos/

Comando	O que faz
git init	Cria um repositório novo na pasta atual
git clone <url>	Copia um repositório remoto para o seu PC
git status	Mostra o que foi alterado, preparado ou não rastreado
git add <arquivo>	Move alterações para a área de staging
git commit -m "mensagem"	Registra as alterações preparadas no histórico
git log	Lista o histórico de commits
git diff	Mostra as diferenças entre versões

/Principais comandos/

Comando	O que faz
git branch	Lista as branches existentes
git branch <nome>	Cria uma nova branch
git checkout <nome>	Troca para outra branch
git merge <nome>	Mescla uma branch na atual

/Principais comandos/

Comando	O que faz
git pull	Baixa e aplica as alterações do remoto
git push	Envia seus commits locais para o remoto

/Mono vs Poly/

- Monorepo → tudo num repositório só
- Polyrepo → cada parte num repositório separado, dev acessa o que precisa, não tudo.
- Não existe certo e errado, é decisão de arquitetura